

JPA + Entity + Repository 간단 예제

Spring Boot Java와 Kotlin 비교

구분	Java + Spring Boot	Kotlin + Spring Boot
Entity 선언	@Entity public class Member	@Entity class Member
ID	private Long id	var id: Long? = null
이름	private String name	var name: String = ""
Getter/Setter	직접 작성 또는 도구 활용	var 사용 시 별도 작성 불필요
Repository	extends JpaRepository	: JpaRepository
저장	repository.save(member)	repository.save(member)
전체 조회	repository.findAll()	repository.findAll()
한 건 조회	repository.findById(1L)	repository.findById(1L)
삭제	repository.deleteById(1L)	repository.deleteById(1L)
특징	명시적이고 코드가 비교적 김	간결하고 코드량이 적음

Java Entity

```
@Entity
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    // Getter / Setter
}
```

java

Kotlin Entity

```
@Entity
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = ""
)
```

kotlin

Java Repository

```
public interface MemberRepository
    extends JpaRepository<Member, Long> {
}
```

java

Kotlin Repository

```
interface MemberRepository :  
    JpaRepository<Member, Long>
```

kotlin

기본 CRUD

```
save()      → 저장  
findAll()   → 전체 조회  
findById()  → 한 건 조회  
deleteById() → 삭제
```

text

핵심 관계

JPA → Entity → Repository → MariaDB

JPA = 객체와 DB의 영속성을 관리하는 표준

Entity = DB 테이블을 객체로 표현

Repository = Entity를 저장·조회·삭제하는 DB 작업 창구

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.